

Deep Learning for Solving Dynamic Models

Session 9: Economics-informed neural networks, economic and financial applications

Simon Scheidegger

HEC, University of Lausanne

Based on Raissi, Perdikaris & Karniadakis (2019), *Journal of Computational Physics* 378, 686–707

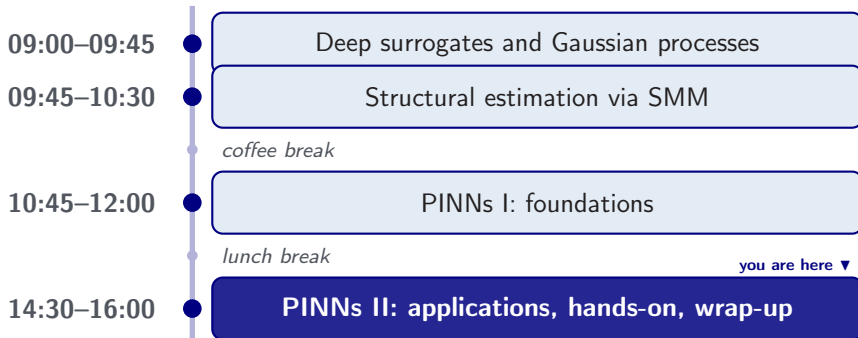
University of Padova

September 24, 2026

Further: Sirignano & Spiliopoulos (2018), Deep Galerkin Method for PDEs in high dimensions.

Al-Aradi, Correia, Naez, Hoshisashi & Schwab (2018), DGM PDE solver.

Day 2 at a Glance



Outline of the PINN Block, Sessions 8 and 9

1. From DEQNs to PINNs

Bridging discrete and continuous time; the PINN loss.

2. PINN Fundamentals

Automatic differentiation for PDEs, collocation, training loop.

3. Soft vs. Hard Boundary Conditions

Trial solutions, 2D Poisson, practical guidance.

4. Economic Applications: HJB Equation

Cake-eating problem, scaled trial solution + hard BCs, Adam $\rightarrow$ L-BFGS, analytical benchmark; DGM as optional high-D architecture.

5. Financial Applications: Black–Scholes

Option pricing via PINNs, comparison with analytical formula.

6. When to Use One, and What Goes Wrong

PINNs vs. grid solvers, inverse problems, failure modes, and which notebooks to run.

Roadmap of This Session (90 min)

Part II of two. Session 8 built the method: a PDE residual on collocation points, and a deliberate choice between soft and hard boundary conditions. Now we put it to work.

IV. HJB equations (20 min)

- ▶ Cake-eating: scaled trial solution, hard BCs, closed-form benchmark
- ▶ DGM as an optional high-dimensional architecture

V. Finance: Black–Scholes (8 min)

- ▶ Option pricing against the closed form; Greeks for free by autodiff

VI. When to use one (12 min)

- ▶ A borrowing constraint, where the grid wins and the residual does not tell you
- ▶ Inverse problems: ρ recovered from data; the multi-component loss, again

Hands-on (20 min): build a PINN from scratch, at the keyboard.

Wrap-up (30 min): what to use when, where to go next, and closing questions.

Part IV

Economic Applications: The HJB Equation

The Hamilton–Jacobi–Bellman Equation

Generic continuous-time optimization:

$$\rho V(\mathbf{x}) = \max_c \{u(c) + V_{\mathbf{x}} \cdot f(\mathbf{x}, c)\}$$

- ▶ $V(\mathbf{x})$: **value function** (unknown)
- ▶ ρ : discount rate
- ▶ $u(c)$: flow utility
- ▶ $f(\mathbf{x}, c)$: law of motion for state $\mathbf{x}$
- ▶ $V_{\mathbf{x}} = \nabla V$: gradient of the value function

This is a PDE! First-order (or second-order with diffusion).

⇒ **Perfect fit for PINNs**: the network approximates V , autograd gives $V_{\mathbf{x}}$, and the HJB residual is the loss.

Cake-Eating Problem: Setup

A household with wealth $a(t)$ chooses consumption $c(t)$ to maximize lifetime utility:

$$\max_{\{c(t)\}} \int_0^\infty e^{-\rho t} \frac{c(t)^{1-\gamma}}{1-\gamma} dt \quad \text{s.t.} \quad \dot{a} = r a - c$$

HJB equation: $\rho V(a) = \max_c \left\{ \frac{c^{1-\gamma}}{1-\gamma} + V'(a)(r a - c) \right\}$

FOC: $c^{-\gamma} = V'(a) \Rightarrow c^* = (V'(a))^{-1/\gamma}$.

Analytical solution (with $\kappa = \frac{\rho - (1-\gamma)r}{\gamma} > 0$):

$$V^*(a) = \frac{\kappa^{-\gamma}}{1-\gamma} a^{1-\gamma}, \quad c^*(a) = \kappa a$$

Parameters: $\gamma = 2$, $\rho = 0.05$, $r = 0.03 \Rightarrow \kappa = 0.04$.

PINN for the HJB Equation: Hard-BC Trial Solution

Anchor + mask + scaled MLP. Let $x(a) = (a - a_{\min}) / (a_{\max} - a_{\min}) \in [0, 1]$ on $[a_{\min}, a_{\max}] = [0.5, 5]$, and $S = |V^*(a_{\min})|$:

$$\hat{V}(a) = V^*(a_{\min}) + x(a) [V^*(a_{\max}) - V^*(a_{\min})] + x(a)(1 - x(a)) S f_{\theta}(x(a)).$$

Endpoints are exact *by construction*, so there is no boundary loss. The HJB is first order, so one value pins the solution; wealth flows towards $a_{\min}$ ($\dot{a} = -0.01 a$), which is where the anchor matters.

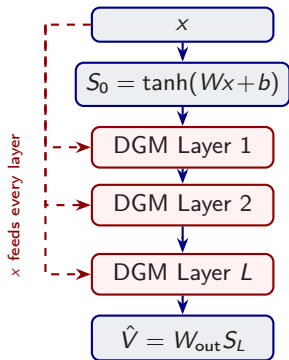
Consumption via FOC, $\hat{c}(a) = (\hat{V}'(a))^{-1/\gamma}$, with softplus safeguard on $\hat{V}'$.

Loss: only the interior HJB residual remains,

$$\ell = \frac{1}{N} \sum_i \mathcal{R}(a_i)^2, \quad \mathcal{R}(a) = \rho \hat{V} - \left[\frac{\hat{c}^{1-\gamma}}{1-\gamma} + \hat{V}'(ra - \hat{c}) \right].$$

Scale: V^* runs from -1250 to -125 on this domain. The factor S lets the network work in units of order one; the residual is still of order $\rho V \sim 16$, which is why the Adam loss starts in the hundreds.

Optional Architecture for High-D PDEs: DGM



Deep Galerkin Method

(Sirignano & Spiliopoulos, 2018)

- Pays off only in **higher dimensions**, or where the PDE has sharp features.
- The input x enters **every** layer, not just the first, that is what resolves steep gradients.
- Each layer is **gated**, LSTM-style; the x -skips recall Highway Networks.

Not the default choice

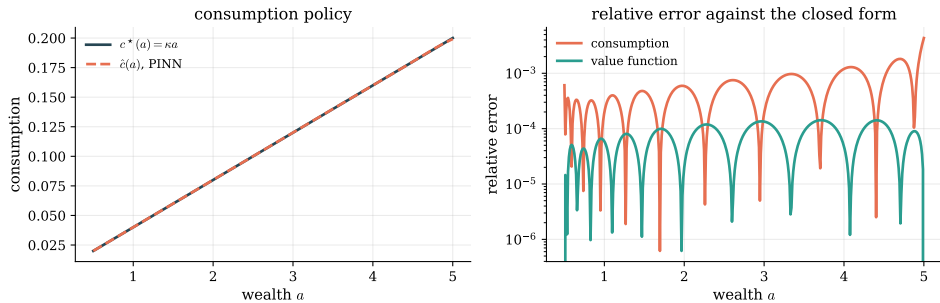
For the 1D cake problem, the trial-solution MLP of the previous slide is simpler *and* more accurate.

HJB Residual Implementation

```
def consumption_and_residual(V_of, a, rho_val=rho):  
    a = a.clone().requires_grad_(True)  
    V = V_of(a) # trial solution  
    V_a = torch.autograd.grad(V, a, #  $V'(a)$  by autograd  
        grad_outputs=torch.ones_like(V), create_graph=True)[0]  
    V_a = softplus(V_a) + 1e-8 # positivity guard  
    c = V_a ** (-1.0 / gamma) # FOC: optimal c  
    R = rho_val * V - (c**(1-gamma)/(1-gamma) + V_a * (r*a - c))  
    return c, R # policy, HJB residual
```

The softplus safeguard prevents $V'(a)^{-1/\gamma}$ from blowing up if $V'(a) \leq 0$ during early training.
`rho_val` is an argument because the inverse problem, two frames on, makes it a trainable tensor.

Cake-Eating: Against the Closed Form



Stored run of 09_01: 2,000 Adam steps on 512 resampled wealth levels, then L-BFGS on 300 fixed points, in float64. Consumption within 4.3×10^{-3} of κa everywhere (relative L^2 error 1.1×10^{-3}), the value function within 1.4×10^{-4} ; the HJB residual is 10^{-2} against $\rho V \sim 16$. The closed form is $V^*(a) = -625/a$, $c^*(a) = 0.04 a$.

Notebook: 09_01_Cake_Eating_HJB_PINN

Extensions: From Cake-Eating to Frontier Models

The PINN/DGM framework scales naturally:

1. **Income risk:** add diffusion $da = (ra - c + y)dt + \sigma dW \Rightarrow$ second-order HJB
2. **Aiyagari-type models:** HJB + Kolmogorov Forward Equation $\Rightarrow$ coupled PDE system
3. **General equilibrium:** prices clear markets $\Rightarrow$ hybrid DEQN + PINN
4. **High dimensions:** multiple assets, heterogeneous agents $\Rightarrow$ no grid needed!

See: Achdou et al. (2022), Fernandez-Villaverde et al. (2024), Gopalakrishna (2024).

Part V

Financial Applications: Black–Scholes

The Black–Scholes PDE

European call option (S : asset, K : strike, T : maturity):

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0$$

Terminal: $V(S, T) = \max(S - K, 0)$

Boundary conditions:

- ▶ $V(0, t) = 0$
- ▶ $V(S_{\max}, t) = S_{\max} - Ke^{-r(T-t)}$

PINN approach:

- ▶ Network $\mathcal{N}_\theta(S, t) \approx V(S, t)$
- ▶ Autograd: V_t, V_S, V_{SS}
- ▶ 4 loss terms:

ℓ_{PDE}	interior residual
ℓ_{BC_0}	$V(0, t) = 0$
ℓ_{term}	$V(S, T) = \text{payoff}$
$\ell_{\text{BC}_{S_{\max}}}$	far-field BC

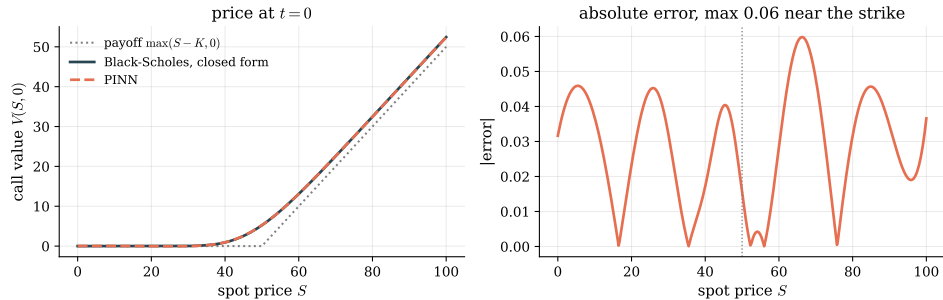
Black–Scholes PINN: Key Code

```
def pde_residual(model, S, t):  
    S.requires_grad_(True)  
    t.requires_grad_(True)  
    X = torch.cat([S, t], dim=1)  
    V = model(X)  
  
    V_t = torch.autograd.grad(V, t, ...)[0]  
    V_S = torch.autograd.grad(V, S, ...)[0]  
    V_SS = torch.autograd.grad(V_S, S, ...)[0]  
  
    # Black-Scholes PDE residual  
    res = V_t + 0.5*sigma**2*S**2*V_SS + r*S*V_S - r*V  
    return res
```

Loss:

$$\ell = \text{MSE}(\text{res}) + \text{MSE}(V|_{S=0}) + \text{MSE}(V|_{t=T} - \text{payoff}) + \text{MSE}(V|_{S_{\max}} - \text{target})$$

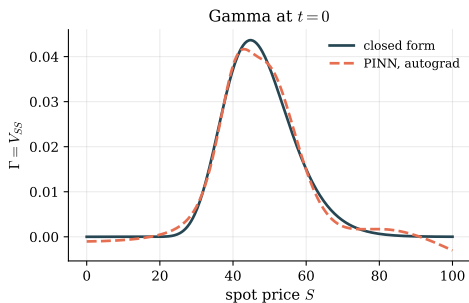
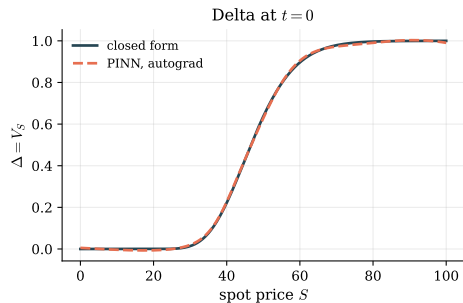
Black–Scholes: Against the Closed Form



$K = 50$, $T = 1$, $r = 0.05$, $\sigma = 0.2$, $S_{\max} = 100$. Network $2 \rightarrow 64 \rightarrow 64 \rightarrow 64 \rightarrow 1$, tanh; 2,500 Adam steps, then L-BFGS on one fixed design. At $t = 0$ the price is within 0.06 of the closed form on $[0, 100]$ (mean 0.03); at the money 5.24 against 5.23. The largest error sits above the strike, where the terminal payoff has its kink. The gap between payoff and price is the **time value**.

Notebook: 09_02_Black_Scholes_PINN

The Greeks Come Out of the Same Network

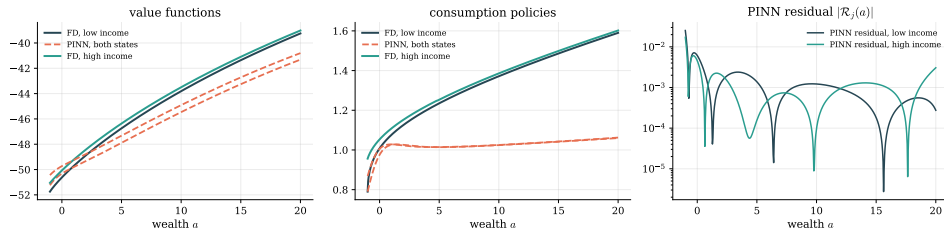


Delta $= V_S$ and Gamma $= V_{SS}$ are the first and second derivative of the network, the same autograd calls the residual used. No bumping, no re-solve. Delta is within 0.01 of the closed form; Gamma, a second derivative of a network trained on a residual that itself contains V_{SS} , within 0.003 of a peak of 0.044.

Part VI

When to Use One, and What Goes Wrong

Should You Use a PINN at All? Not Here



09_03: a household with a borrowing constraint and a two-state income, partial equilibrium, no closed form. The upwind finite-difference scheme converges in 13 iterations. The PINN, same economics, state constraint as an inequality at $\underline{a}$, ends at a loss of 4×10^{-6} and value functions within 3%, and a consumption policy **26 % off**: it consumes about its income at every wealth level, where the reference household with 20 units of wealth and $\rho > r$ dissaves 0.6 a year. The residual, 10^{-3} , does not say so. The benchmark does.

Why It Fails Here, and Where the Trade Flips

Why. The HJB is first order and its derivative enters multiplied by savings:

$$\rho V_j = u(c_j) + V_j' \underbrace{(ra + wz_j - c_j)}_{s_j} + \lambda_{jk}(V_k - V_j).$$

- ▶ Small s_j switches the derivative off, and many smooth V nearly satisfy what is left: the residual can hardly tell the optimal policy from one that stays put.
- ▶ The upwind scheme propagates values **along the flow**, which selects the constrained viscosity solution. A residual has no notion of direction.
- ▶ Cake-eating worked because its endpoints were anchored; the inverse problem because the data anchored it.

Where the trade flips

A tensor grid needs n^d points. At $n = 50$:
 $d = 1$: 50; $d = 2$: 2,500; $d = 4$: 6.3 million; $d = 6$: 1.6×10^{10} . Collocation cost is set by the sample size you choose, not by d .

Use a PINN when the state space is high-dimensional, the equation is second order (a diffusion keeps the derivatives in play everywhere), the domain is irregular, or you need the solution *and* its derivatives as one differentiable object. On a 1-D first-order HJB with a state constraint, use the grid.

The Other Half: Inverse Problems

So far the PDE was known and we solved it. Raissi et al. (2019) also treat the reverse.

Forward problem

Parameters μ given, solve for u .

$$\ell(\theta) = \|\mathcal{D}_\mu[\mathcal{N}_\theta]\|^2 + \text{BCs}$$

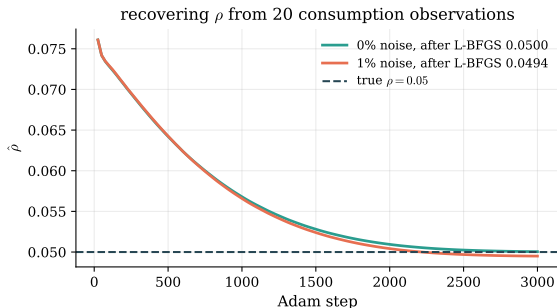
Inverse problem

Some data on u observed, solve for u and μ .

$$\ell(\theta, \mu) = \|\mathcal{D}_\mu[\mathcal{N}_\theta]\|^2 + \|\mathcal{N}_\theta - u^{\text{obs}}\|^2$$

- ▶ μ is just **another set of trainable parameters**: one extra tensor, same optimizer, same loop.
- ▶ In economics μ is a **structural parameter**, risk aversion, a discount rate, an adjustment cost, and the data term is a moment condition.
- ▶ Estimation and solution in **one** optimization, rather than a solver nested inside an estimation loop, as in Session 7 this morning.

The Inverse Problem, Run: Recover ρ From Data



Cake-eating with ρ unknown and consumption observed at 20 wealth levels:

$$\ell(\theta, \rho) = \left\| \frac{\mathcal{R}(\cdot; \theta, \rho)}{\rho_0 S} \right\|^2 + \left\| \frac{\hat{c}(a_k) - c_k^{\text{obs}}}{c_k^{\text{obs}}} \right\|^2$$

- $\log \rho$ is one more trainable tensor; same Adam $\rightarrow$ L-BFGS loop.
- The data replace the boundary conditions: the network for V is free.
- From $\rho_0 = 0.08$: $\hat{\rho} = 0.0500$ with exact data, 0.0495 with 1% noise.

Notebook: 09_01, Section 4

When PINNs Fail: the Multi-Component Loss, Again

The soft-BC loss is a weighted sum of residuals that live at different scales:

$$\ell_{\theta} = \underbrace{\lambda_{\text{pde}} \ell_{\text{pde}}}_{\text{interior}} + \underbrace{\lambda_{\text{bc}} \ell_{\text{bc}}}_{\text{boundary}} + \underbrace{\lambda_{\text{ic}} \ell_{\text{ic}}}_{\text{initial}}$$

This is exactly the Day 1 pathology.

- ▶ A residual carrying $S^2 \partial_{SS} V$ is not in the same units as one carrying V itself.
- ▶ The largest-scale component captures the gradient; the others are effectively ignored.
- ▶ Adaptive weights cannot close a gap of several orders of magnitude.

What to do, in order

1. **Non-dimensionalize** first, rescale so every residual is $O(1)$.
2. Enforce **structurally** where you can: a hard-BC trial solution deletes ℓ_{bc} from the sum.
3. Only then reach for adaptive weights.
4. **Keep a benchmark**: in 09_03 the loss fell to 4×10^{-6} with the policy 26 % wrong.

In Session 3 a change of units cut the total relative error from 0.861 to 0.011, a factor 75, where the best adaptive rule reached 0.448 (03b_Loss_Balancing.pdf). See Wang, Teng & Perdikaris (2021).

Hands-On: What to Run

All seven notebooks are PyTorch, CPU-only, and self-contained. Run them on nuvolos.cloud, no local setup needed.

Session 8, the method

- ▶ The simplest possible PINN: $y'' = -1$ with zero BCs 08_01
- ▶ Soft penalty vs. hard trial solution, the core choice 08_02
- ▶ 2D Poisson: hard BCs by transfinite interpolation 08_03

Session 9, the applications

- ▶ The first economic application: the cake-eating HJB 09_01
- ▶ Black–Scholes, Greeks for free by autodiff 09_02
- ▶ PE HJB with discrete income, against an FD benchmark 09_03

Now: build one yourself 09_04

A fill-in-the-blank PINN for $u'' + u = 0$ on $[0, \pi/2]$, $u(0) = 0$, $u(\pi/2) = 1$, solution $\sin x$; 20 minutes, solutions follow each task. Bonus at home: hard BCs, and why the same equation on $[0, \pi]$ with zero boundary values is ill-posed: every $c \sin x$ solves it, and the network returns $u \equiv 0$ at a loss of 10^{-6} .

Summary and Key Takeaways

Method

PINNs embed **differential equations** directly into the neural network loss function via **automatic differentiation**.

Boundary Conditions

Hard BCs (trial solutions) eliminate tuning and ensure exact satisfaction. Use **soft BCs** for complex domains.

Applications

HJB equations, Black–Scholes, and any continuous-time model. Scales to **high dimensions** without grids.

From DEQNs to PINNs:

- ▶ **Same philosophy:** economic/financial equations *are* the loss
- ▶ **New ingredient:** `torch.autograd.grad` for derivatives of $\mathcal{N}_\theta$
- ▶ **Broader scope:** any PDE, not just algebraic equilibrium conditions

Questions?

Simon Scheidegger

HEC, University of Lausanne

<https://sischei.github.io>

Materials:

github.com/sischei/deep_learning_padova_0926

Companion Notebooks:

08_01	Smallest complete PINN
08_02	Soft vs. hard BCs
08_03	2D Poisson
09_01	Cake-eating, inverse
09_02	Black-Scholes
09_03	FD vs. PINN
09_04	Build a PINN

Next up:

Hands-on, then
the course wrap-up